

树上倍增

这几年倍增算法特别热门，几乎年年都考，而且越考越难，由以前的简单的求lca到现在的现在倍增预处理然后跑DP等等，无论是NOIP还是省选甚至NOI，倍增都是最热门的一个考点。

NOIP	2019	树的重心
NOIP	2018	保卫王国
NOIP	2016	天天爱跑步
NOIP	2015	运输计划
NOIP	2013	货车运输
NOIP	2012	疫情控制
NOIP	2012	开车旅行

树上倍增

树上倍增就是用来求解不带修改的树上查询问题，他的原理和RMQ一致，都是使用倍增跳跃的思想，这样就能把一步一步向上跳跃 $O(n)$ 的时间复杂度优化为 $O(\log n)$ 。

在RMQ中， $f[i][j]$ 表示以 i 点为起点，长度为 2^j 的区间 $[i, i+2^j-1]$ ，那么该区间可以由 $f[i][j-1]$ 和 $f[i+2^{j-1}-1][j-1]$ 合并而来。

同样，在树上倍增中， $f[i][j]$ 表示 i 点向上跳跃 2^j 步的位置，那么 $f[i][j]$ 可以由 $f[f[i][j-1]][i-1]$ 得到，也就是说深度越深的点可以由深度较浅的点转移过来。这样我们只需要预处理 $i * \log(\text{deep})$ 的数组就可以存储所有点向上跳跃 2^j 的步数的位置。

树上倍增

```
void dfs(int x)
{
    book[x]=true;
    for(int i=1;(1<<i)<=deep[x];i++)//求出该点往上2^k的点的位置。
    {
        fa[x][i]=fa[fa[x][i-1]][i-1];
    }
    for(int i=head[x];i;i=edge[i].next)//深度优先遍历
    {
        int u=edge[i].to;
        if(!book[u])
        {
            fa[u][0]=x;//x是u的父亲
            deep[u]=deep[x]+1;//u的深度是x的深度+1
            dfs(u);
        }
    }
}
```

树上倍增

预处理好之后就是查询任意两点的最近公共祖先，最大的问题就是很有可能最近公共祖先所在的深度和两点的深度之差不是刚好等于 2^i 。

但是我们知道对于任意一个十进制数都可以拆分成一个唯一的二进制数，那么对于任意深度的点，都可以通过一系列 2^i 最终到达深度比他浅的点，只是需要跳几次而已，并且跳跃次数不会多于 $\log n$ 。所以我们只需要判断两个点向上跳知否可以跳到同一个点就可以了，为了方便，我们先把这两个点放在同一层，每次跳跃相同的高度，因为一旦跳到根结点以及根结点之上，那么祖先肯定是相等，如果不相等，说明肯定可以跳上去，并且可以先同时跳到该位置，再继续跳。

注意最开始要特判两个点是否相等。

树上倍增

```
for(int i=0;(1<<i)<=t;i++)//先调整到同一高度, 即把u向上调整t个高度
{
    if((1<<i)&t)//二进制数, 如果该位有1就跳
    {
        u=fa[u][i];//一定要更新u。不然全错
    }
}
if(u==v)//如果u,v在一条链上, 直接输出上面那个店
return u;
for(int i=20;i>=0;i--)//i可以等于0, 表示其父亲结点
{
    if(fa[u][i]!=fa[v][i])
    {
        u=fa[u][i];//更新
        v=fa[v][i];
    }
}
return fa[u][0];//u和v是最后一个不相等的结点其父亲结点是相等的, 所
```

树上倍增

倍增求解两点之间的距离：

对于一棵树上的任意两点，直接求解他们的距离比较麻烦，我们可以把这条链分成两段，分别求解这两个点到他们的lca的距离。

如果边权相同，我们可以直接变跳跃边累加，但是如果边权不相等呢？跳跃的次数并不等于跳跃的距离。

我们仍然可以考虑前缀和的思想，记录每一个点到根结点的距离 $d[i]$ ，那么对于两点 $[x, y]$ 之间的距离就变成了 $d[x] + d[y] - 2 * d[lca(x, y)]$ 。

树上差分

明白了树上倍增的原理就知道，先预处理再查询，所以是不支持修改的，只要任意一个点发生变化，这条链上的所有点的祖先都会发生变化，那么修改的时间复杂度是比较高的。

但是对于某一种修改：

每次修改一条链，最后只有一次查询(少量)

我们可以参考种树问题的差分思路来求解。

也就是说如果种树的那一条是一棵树，每次挖去的是两个点之间的简单路径的树，最后问还剩下多少棵树，应该怎么做？

树上差分

题目描述

展开

松鼠的新家是一棵树，前几天刚刚装修了新家，新家有 n 个房间，并且有 $n-1$ 根树枝连接，每个房间都可以相互到达，且两个房间之间的路线都是唯一的。天哪，他居然真的住在“树”上。

松鼠想邀请小熊维尼前来参观，并且还指定一份参观指南，他希望维尼能够按照他的指南顺序，先去 a_1 ，再去 a_2 ，……，最后到 a_n ，去参观新家。可是这样会导致维尼重复走很多房间，懒惰的维尼不停地推辞。可是松鼠告诉他，每走到一个房间，他就可以从房间拿一块糖果吃。

维尼是个馋家伙，立马就答应了。现在松鼠希望知道为了保证维尼有糖果吃，他需要在每一个房间各放至少多少个糖果。

因为松鼠参观指南上的最后一个房间 a_n 是餐厅，餐厅里他准备了丰盛的大餐，所以当维尼在参观的最后到达餐厅时就不需要再拿糖果吃了。

输入格式

第一行一个整数 n ，表示房间个数
第二行 n 个整数，依次描述 a_1-a_n

接下来 $n-1$ 行，每行两个整数 x, y ，表示标号 x 和 y 的两个房间之间有树枝相连。

输出格式

一共 n 行，第 i 行输出标号为 i 的房间至少需要放多少个糖果，才能让维尼有糖果吃。

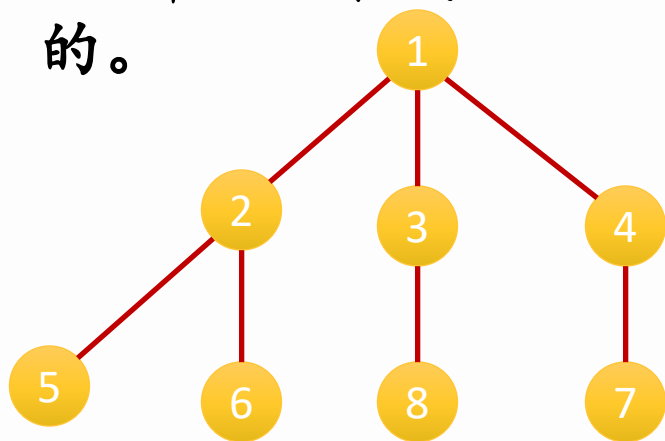
树上差分

按照前面差分的思想，我们要把一条链的修改转换成单点修改，把树上一个点的权值转换成前缀权值。

直接把任意两个点之间的简单路径换成前缀会比较麻烦，所以我们把一条路径看做两部分。

比如 $a \rightarrow b$, $\text{lca}(a, b) = c$, 那么我们可以看成两条链 $a \rightarrow c$, $b \rightarrow c$ 。

那么如果我们按照这种方式进行差分，明显是有问题的。



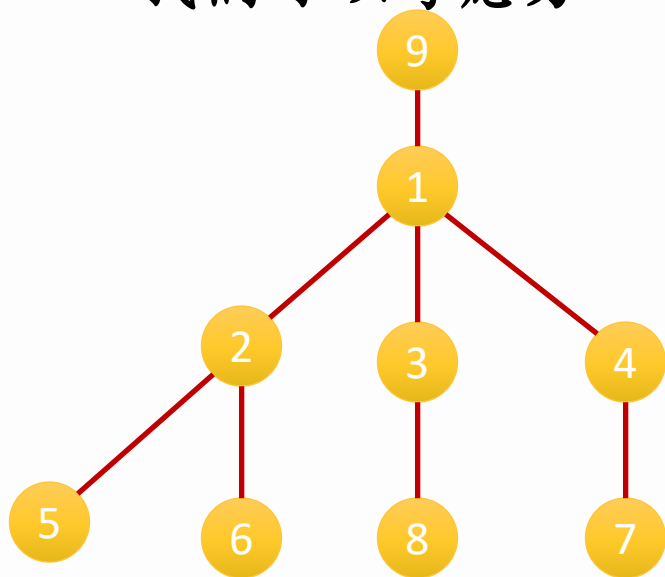
比如修改链 $2 \rightarrow 4$, 那么就是 $a[1]++$, $a[5]--$, $a[1]++$, $a[7]--$, 这样修改过的链就是正确的, 但是对于其他的链比如 $1 \rightarrow 8$ 就错误了, 所以我们不能这样差分。

树上差分

差分的方式有很多种，只要差分之后能保证一个点的权值等于差分数组部分之和就可以了，并且是有规律的。

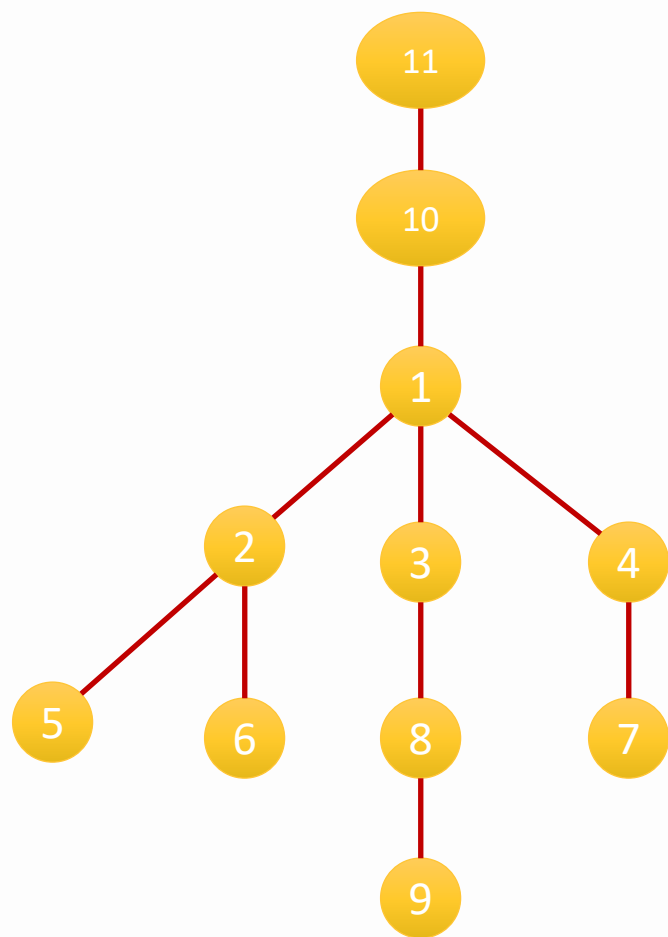
既然从上面往下面差分有一点问题，那么我们可以尝试一下从下面往上面差分。

我们可以考虑另一种差分方式：把子树之和当做点权



比如修改链 $2 \rightarrow 4$ ，那么就是 $a[2]++$, $a[4]++$ ，我们再考虑他们的最近公共祖先，该位置的子树 $+2$ ，但是实际只加了1，所以 $a[1]--$ ，但是1的祖先9并没有改变，所以 $a[9]--$ ，我们再看其他的，3的子树并没有改变，2,4的儿子也没有改变

树上差分



因为是差分+前缀和的思想，所以对于每一个点，我们可以看做子树之和。假设我修改一条链 $2 \rightarrow 4$ ，那么为了让2的子树不受影响，并且最终2的答案只减少1，那么我们只需要修改 $cf[2]$ ，4也同理，那么不会影响到2,4的子树和其他链的，但是祖先1的子树之和-2，实际只减少了1，那么 $cf[1]+1$ 才能保持不变。我们再考虑整棵子树，1所在树总的数量-1,1上面的点统计答案的时候也会减少，所以1的祖先 $cf[10]+1$ 。这样该减的就减掉了，其他的都不受影响。

树上差分

第一步：树上倍增预处理祖先

第二步：根据链的修改处理差分数组。

```
for(int i=2;i<=n;i++)  
{  
    int pa=lca(a[i-1],a[i]);  
    cf[a[i-1]]++;  
    cf[a[i]]++;  
    cf[pa]--;  
    cf[fa[pa][0]]--;  
}
```

第三步：dfs统计子树答案。



the merchant

简化题意

给你一棵树，点有点权，到达一个点你可以花费该点的点权买入一个东西，然后在另一个点把这个东西卖出（卖出的时候手上必须有东西），只能买入卖出一次，问你从一个点 u 到一个点 v 的路径上能获得的最大收益。



the merchant

一定要注意该题有方向性，并不是直接求一条链的最大值和最小值相减就可以了，因为你卖这个物品之前必须先买这个物品。

所以用树上倍增求解就有三种情况：

从A→B：

A链上买，A链上卖

A链上买，B链上卖

B链上买，B链上卖

首先，我们要维护一个 $\max[i][j]$ 和 $\min[i][j]$ ，表示从i点向上跳 2^j 的路径上的最大/最小值

由于在一条链上可能从下到上，也可能从上到下，所以我们需要记录在链上up和down两种情况的最大收益

货车运输

A 国有 n 座城市，编号从 1 到 n ，城市之间有 m 条双向道路。每一条道路对车辆都有重量限制，简称限重。

现在有 q 辆货车在运输货物，司机们想知道每辆车在不超过车辆限重的情况下，最多能运多重的货物。

输入格式

第一行有两个用一个空格隔开的整数 n, m ，表示 A 国有 n 座城市和 m 条道路。

接下来 m 行每行三个整数 x, y, z ，每两个整数之间用一个空格隔开，表示从 x 号城市到 y 号城市有一条限重为 z 的道路。

注意： $x \neq y$ ，两座城市之间可能有多条道路。

接下来一行有一个整数 q ，表示有 q 辆货车需要运货。

接下来 q 行，每行两个整数 x, y ，之间用一个空格隔开，表示一辆货车需要从 x 城市运输货物到 y 城市，保证 $x \neq y$

货车运输

该题最开始是一幅图，如果要保证连通的话一棵树就够了，并且每条边都有一个载重，所以我们可以想到有些路是可以不用走的，哪些路呢？边权载重尽可能小的路。

所以我们优先选择载重最大的边，把这些点连成一棵树，所以我们可以先用kruskal跑一遍最大生成树

然后对于任意两点，我们需要求出这两点之间的边权的最小载重，这个直接树上倍增处理就可以了。

运输计划

公元2044年，人类进入了宇宙纪元。

L 国有 n 个星球，还有 $n - 1$ 条双向航道，每条航道建立在两个星球之间，这 $n - 1$ 条航道连通了 L 国的所有星球。

小 P 掌管一家物流公司，该公司有很多个运输计划，每个运输计划形如：有一艘物流飞船需要从 u_i 号星球沿最快的宇航路径飞行到 v_i 号星球去。显然，飞船驶过一条航道是需要时间的，对于航道 j ，任意飞船驶过它所花费的时间为 t_j ，并且任意两艘飞船之间不会产生任何干扰。

为了鼓励科技创新，L 国国王同意小 P 的物流公司参与 L 国的航道建设，即允许小 P 把某一条航道改造成虫洞，飞船驶过虫洞不消耗时间。

在虫洞的建设完成前小 P 的物流公司就预接了 m 个运输计划。在虫洞建设完成后，这 m 个运输计划会同时开始，所有飞船一起出发。当这 m 个运输计划都完成时，小 P 的物流公司的阶段性工作就完成了。

如果小 P 可以自由选择将哪一条航道改造成虫洞，试求出小 P 的物流公司完成阶段性工作所需要的最短时间是多少？

运输计划

一定要注意，该题不是求航行经过次数最多的边，而是求改造之后的最大的运输计划所用的时间最少。

所以我们很容易想到二分答案，加下来考虑如何验证。

我们假设的运输所花的最大时间为 x ，那么我们需要在树上找到一条或者多条公共边满足 $\text{Max-len} \leq x$

接下来我们要考虑哪些边长度大于 x ，树上求两点之间的路径的方法很简单，前面已经讲个，用倍增预处理出来任意一个点到起点的路径，然后加起来 $-2 * \text{lca}$ 就可以了。

对于长度大于 x 的路径，我们标记一下，看下这些线段是否存在一条公共边。可以写树链剖分+线段树，但是该题只有一个询问，也可以直接树上差分来做。



树上倍增

这几道题都是比较裸的树上倍增问题，主要就是询问一个最近公共祖先，或者求路径的最大值。

而最近几年的考题逐渐和动态规划扯上了关系，用倍增来优化动态规划，用倍增维护一个信息，而和最近公共祖先没有关系，就有点类似于RMQ问题。



开车旅行

小 A 和小 B 决定利用假期外出旅行，他们将想去的城市从 1 到 n 编号，且编号较小的城市在编号较大的城市的西边，已知各个城市的海拔高度互不相同，记城市 i 的海拔高度为 h_i ，城市 i 和城市 j 之间的距离 $d_{i,j}$ 恰好是这两个城市海拔高度之差的绝对值，即 $d_{i,j} = |h_i - h_j|$ 。

旅行过程中，小 A 和小 B 轮流开车，第一天小 A 开车，之后每天轮换一次。他们计划选择一个城市 s 作为起点，一直向东行驶，并且最多行驶 x 公里就结束旅行。

小 A 和小 B 的驾驶风格不同，小 B 总是沿着前进方向选择一个最近的城市作为目的地，而小 A 总是沿着前进方向选择第二近的城市作为目的地（注意：本题中如果当前城市到两个城市的距离相同，则认为离海拔低的那个城市更近）。如果其中任何一人无法按照自己的原则选择目的城市，或者到达目的地会使行驶的总距离超出 x 公里，他们就会结束旅行。

在启程之前，小 A 想知道两个问题：

1、 对于一个给定的 $x = x_0$ ，从哪一个城市出发，小 A 开车行驶的路程总数与小 B 行驶的路程总数的比值最小（如果小 B 的行驶路程为 0，此时的比值可视为无穷大，且两个无穷大视为相等）。如果从多个城市出发，小 A 开车行驶的路程总数与小 B 行驶的路程总数的比值都最小，则输出海拔最高的那个城市。

2、 对任意给定的 $x = x_i$ 和出发城市 s_i ，小 A 开车行驶的路程总数以及小 B 行驶的路程总数。

开车旅行

首先，预处理出来离每个点最近和第二近的在他右边的点是哪一个。

肯定不能用 n^2 的做法，可以考虑数据结构，找每一个点右边第一个第二个比他大或者小的点，不过有点麻烦。

实际上我们可以先按照高度排序，不管方向，排好序之和，我们找初始编号为1的城市的最大/次大点，最多就4种情况， $i-2, i-1, i+1, i+2$ ，因为第一个点在最左边，所以找到的点一定在他右边，然后我们删除第一个点，接下来找编号为2的点，因为第一个点删除了，那么 $[i-2, i+2]$ 中找到的点一定在他右边，然后已知删除查找就可以了，效率更好，代码更短，用双向链表实现。

注意：不能直接排序然后找 $[i-2, i+2]$ ，因为只能到右边，很有可能没在这个区间

开车旅行

因为每一个点作为起点都有可能是答案，所以我们要先枚举起点，但是询问 m 又有多组数据，所以记录 $dp[i][j]$ 表示从 i 出发走 j 步的距离是多少是不行的，我们可以考虑树上倍增的思想，不再是一步一步的走，而是 $dp[i][j]$ 表示从起点出发走 2^j 的位置，因为小A和小B走的点不一样，所以我们要分开处理，记录 $a[i][j]$ 表示单独小A走 2^j 步之后的距离， $b[i][j]$ 表示单独小B走 2^j 步的距离，最后再开一个数据记录从起点走 2^j 步最后到达的点是哪个点，最开始 $a[i][0]$, $b[i][0]$ 就是离起点最近和次近的点

疫情控制

题目描述

展开

H 国有 n 个城市，这 n 个城市用 $n - 1$ 条双向道路相互连通构成一棵树，1号城市是首都，也是树中的根节点。

H 国的首都爆发了一种危害性极高的传染病。当局为了控制疫情，不让疫情扩散到边境城市（叶子节点所表示的城市），决定动用军队在一些城市建立检查点，使得从首都到边境城市的每一条路径上都至少有一个检查点，边境城市也可以建立检查点。但特别要注意的是，首都是不能建立检查点的。

现在，在 H 国的一些城市中已经驻扎有军队，且一个城市可以驻扎多个军队。一支军队可以在有道路连接的城市间移动，并在除首都以外的任意一个城市建立检查点，且只能在一个城市建立检查点。一支军队经过一条道路从一个城市移动到另一个城市所需要的时间等于道路的长度（单位：小时）。

请问最少需要多少个小时才能控制疫情。注意：不同的军队可以同时移动。

疫情控制

一个点如果要控制尽可能多的控制结点那么很明显是向上移动到离根结点最近的点，所以军队每次移动都是向上移动的。

该题问最坏的时间最小是多少，那么直接就二分答案，我们需要记录一个点在我们验证的时间内最多能到达哪个点，那么一般这种要求向上走多少步能到哪的题都是用倍增来处理，所以我们先用树上倍增来维护任意一个点走 2^j 能到达的距离。

那么我们把所有军队在假定时间内到达的位置都求出来，那么这样再跑bfs验证是否可以到根结点就可以了吗？并不是，如果一个点可以到达根结点甚至还可以走，那么我们还要讨论下。

疫情控制

如果一个点可以到达根结点并且还可以继续走下去，那么我们先把这样的点存起来，因为这个点的军队可能去镇守其他路径，如果这条路径军队多了，其他路径没有军队，所以我们开一个数组记录这些可能到达终点的军队，同时，当我们把所有军队都移动之后，还可能有一些能到达叶子结点的点未被控制，那么我们就要用刚才存的这些军队去控制这些点？很多军队很多点怎么控制呢？

很明显贪心，离起点越远的未被控制的点优先用还剩下移动距离最远的点去控制，如果不可以控制就说明不行，所以我们把这些可用军队的剩余距离排序，未被控制的点到起点的距离排序，贪心的去控制



练习题

MSOJ	1331	相遇
POJ	3728	the merchant
POJ	3417	network
洛谷	1967	货车运输
洛谷	2680	运输计划
洛谷	1081	开车旅行
洛谷	1084	疫情控制
洛谷	1600	天天爱跑步

